Operator Guide

Day-to-day operations manual for the Nevada County Experience platform.

Who this is for

Whoever runs the platform week to week — chamber staff, county-IT, or a contractor on retainer. This guide assumes:

- You can use a web browser and File Explorer competently
- You can open a terminal (PowerShell or cmd) and type a command
- You don't need to write code

How to use this guide

Each section is a single, self-contained workflow with copy-paste commands. The Quick Reference page lists the most common tasks. When in doubt, jump to Troubleshooting at the end.

Quick Reference

Goal	Section	Time
Start the local server	Section 1	30 sec
Scrape & approve new events	Section 2 + 3	5 min
Run AI to clean up tags & areas	Section 4	~1 min
Add or edit an experience	Section 5	2 min/entry
Update the live demo on GitHub Pages	Section 6	2 min
Share public feeds with a partner	Section 7	1 min
Stop the server	Section 1c	5 sec
Something's broken	Section 8 (Troubleshooting)	varies
Where do files live?	Section 9 (File map)	reference

1. Starting & stopping the local server

The local Flask server is required for the admin panel, AI Categorize, and live event scraping.

1a. Easiest: double-click the launcher

In the project folder there's a file named `start_server.bat`. **Double-click it.** A console window opens, prints "Running on http://127.0.0.1:5000", and stays open while the server runs.

Tip: right-click `start_server.bat` → Send to → Desktop (create shortcut), so you can launch from the desktop.

1b. From PowerShell (alternative)

```
cd "C:\Users\<your-user>\Documents\nevada-county-experience"
python server.py
```

You should see:

```
============================================================
  Nevada County Experience -- Local Server
  http://127.0.0.1:5000/
  Press Ctrl+C to stop
============================================================
 * Running on http://127.0.0.1:5000
```

1c. To stop the server

In the console window where it's running, press **Ctrl+C** once. The window stays open — you can type the start command again, or just close the window.

1d. Open the site in your browser

Go to **http://localhost:5000** while the server is running. Bookmark it.

2. Scrape new events

Pulls fresh events from 12 active sources — KVMR, NCAC Calendar, Eventbrite, the Chambers, Center for the Arts, Miners Foundry, Crazy Horse Saloon, Golden Era Lounge, The Curious Forge, Wolf Craft Collective, and Nevada County Fairgrounds. Run weekly or whenever you want fresh data.

2a. From the admin panel (easiest)

1. Make sure the server is running (Section 1)
2. Open http://localhost:5000 in your browser
3. Click ■ **Manage Database** in the top-right corner
4. Click the ■ **Events Queue** tab
5. Click the green ■ **Update Now** button (top right)
6. Wait 30-90 seconds — the status bar shows "■ Scraping in progress…"
7. When it finishes, switch to the **Pending** tab — new events appear there

2b. From the command line (advanced)

```
cd "C:\Users\<your-user>\Documents\nevada-county-experience"
python scraper\event_scraper.py

# Or scrape a single source:
python scraper\event_scraper.py --site "Nevada City Chamber"
```

What gets scraped

Source	Method	Typical volume
NCAC Calendar	Trumba JSON feed	~580 events
KVMR	Tribe Events RSS feed	~490 events
The Curious Forge	WooCommerce Store API	~65 class sessions
Crazy Horse Saloon	The Events Calendar REST API	~25 events
Center for the Arts	requests + Selenium fallback	~20 concerts
Eventbrite Nevada	Headless Chrome (React)	~10 events
Nevada City Chamber	Static HTML	~10 events
Wolf Craft Collective	Shopify products.json	~10 craft classes

Source	Method	Typical volume
Golden Era Lounge	Squarespace events JSON	~10 events
Nevada County Fairgrounds	Saffire JSONP (eventsservice.asmx)	~7 fair/festival events
Miners Foundry	Headless Chrome (site 403s requests)	~5-10 events
GV Chamber	Static HTML (Elementor)	~5 events
The Union	RSS first, Selenium fallback	Varies (paywall)

Disabled scrapers (kept in the codebase, off by default): *Go Nevada* and *Go Nevada Festivals* — Cloudflare 403s both requests and headless Chrome. *Nevada Theatre* — MEC plugin archive is AJAX-lazy-loaded and not worth a slow brittle scrape (KVMR already covers it well).

3. Approve, dismiss, prune events

After scraping, new events sit in the Pending queue. Approve to publish; dismiss to hide permanently.

3a. Approve individual events

In the Events Queue → Pending tab:

- ✓ **Approve** button: event becomes visible to the public
- ✕ **Dismiss** button: event hidden permanently. Won't come back even if scraped again.
- Click the event title to open the source page in a new tab

3b. Approve them all at once

When the queue is mostly noise-free (typical after AI categorization flags low-quality items), use bulk approval:

"Approve All" button — approves everything pending whose date is today or in the future. Past-dated pending events are auto-dismissed (prevents stale items from re-appearing).

3c. Filter the queue

Above the table you'll find filter controls:

- **Status:** Pending / Approved / Dismissed (one tab each)
- **Source:** dropdown — see only KVMR, only Eventbrite, etc.
- **Date range:** two date inputs — narrow to a specific window

3d. Pruning past events

The pruner removes:

- Pending or approved events whose date has passed
- Dismissed events older than 60 days (long enough to prevent re-import)

Note: public visitors never see expired events — the site filters them out client-side regardless of pruning. Pruning is queue housekeeping, not a visitor-facing concern.

When pruning runs:

- **Automatically** on every scrape (right before merging fresh events)
- **Manually** via the ■ Prune Past button at the top of the Events Queue

- **Manually** via API — `POST /api/events/prune`

Action item — schedule the scraper: the scraper does not run on a schedule out of the box. Set up a nightly run via Windows Task Scheduler (chamber laptop) or cron / systemd timer (VPS) so the queue stays current and pruning fires automatically. Without a schedule, expired events accumulate in the queue (still hidden from visitors) until someone clicks Update Now or ■ Prune Past.

4. AI Categorize (Claude Haiku)

Cleans up tags, infers venue and area, scores quality. Optional but highly recommended for KVMR events.

4a. One-time setup (do this once)

1. Get an Anthropic API key at **console.anthropic.com**
2. Set a $5/month spending cap in your Anthropic billing settings (safety net)
3. Open `scraper/config.py` in a text editor
4. Add this line at the top: `ANTHROPIC_API_KEY = "sk-ant-..."`
5. Save the file. Restart the server.

4b. Run it

1. Open the admin Events Queue (Manage Database → Events Queue)
2. Click the purple ■ **AI Categorize** button
3. A prompt asks how many events to process — leave blank for all, or enter a number for a test run
4. Status bar shows "■ Categorizing…" for ~1-3 minutes
5. When done, refresh the page — events show refined area, venue, tags

4c. Cost

Run pattern	Per run	Monthly equiv
First-time bulk run (all ~1,240 events)	~$0.80	one-time
Weekly: only new events	$0.10 - $0.30	$0.40 - $1.20/mo
Daily: only new events	$0.02 - $0.05	$0.60 - $1.50/mo

4d. What if you don't want to use AI?

Skip the API key setup. The button greys out to ■ `AI (no key)`. Everything else still works — events just keep their basic scraper-assigned tags.

5. Add or edit an experience

The 164 curated experiences (Empire Mine, Lola Restaurant, B&Bs, campgrounds, etc.) live in an inline-editable table.

5a. Open the editor

1. Server running, browser open at http://localhost:5000
2. Click ■ **Manage Database** top right
3. Stay on the **Experiences** tab (it's the default)

5b. Edit a field

1. Find the row by typing in the search input or sorting by clicking a column header
2. Click directly on the cell you want to change (Name, Description, Hours, etc.)
3. Type the new value
4. Click ■ **Save All** in the toolbar — green flash confirms saved

5c. Add a new entry

1. Click **+ Add Experience** in the toolbar
2. A new blank row appears at the bottom
3. Fill in: Name, Description, Area, Type, Hours, Notes, Tags, Season, URL, Lat, Lng
4. Click ■ **Save All**

Required fields for the entry to show: Name, Area, Type, Tags. Everything else is optional but recommended.

5d. Manage tags

Click ■ **Edit Tags** in the toolbar to open the tag taxonomy editor. Add, rename, or delete tags. Changes propagate to all filtering immediately.

5e. Delete an entry

Click the × button at the right end of the row. Confirms before deleting. Click ■ Save All to persist.

6. Update the live GitHub Pages demo

After scraping fresh events or editing experiences, push the updates so the public demo at **liammlrb-eng.github.io/nevada-county-experiences** reflects them.

6a. The full refresh workflow

```
# 1. Make sure server is running, then in admin panel:
#      • ■ Update Now (scrape fresh events)
#      • Wait for completion
#      • ■ AI Categorize (optional, recommended)
#      • Approve All (or review individually)

# 2. From PowerShell, push the updated files to GitHub:
cd "C:\Users\<your-user>\Documents\nevada-county-experience"
git add scraper_output/events.json
git add index.html                    # if you edited any experiences
git commit -m "events update — <date>"
git push

# 3. Wait ~1 minute. The live demo updates automatically.
```

6b. What gets updated where

What	Where it lives	Pushed to GitHub?
Public events	scraper_output/events.json	Yes
Curated experiences (161)	index.html	Yes
Itinerary suggestions (admin)	managed via Manage Database	Yes
Scraper source URLs	scraper/sources.json	Yes
API keys (Google, Anthropic)	scraper/config.py	NO — gitignored
Local backups, cache files	.bak, __pycache__	NO — gitignored

6c. Cache caveat

GitHub Pages caches static files for ~10 minutes by default. If a visitor has the page open while you push an update, they'll see the old data until they hard-refresh (**Ctrl+F5** on Windows / **Cmd+Shift+R** on Mac). New visitors get the latest immediately. Not a problem for tourism use — visitors don't refresh every minute.

6d. Recommended cadence

When	What
Just before peak season (Apr, Oct)	Full scrape + AI + manual review + push
Weekly during season	Scrape + Approve All + push
Monthly off-season	Scrape + push
After major content update	Edit experiences locally + push

7. Public feeds — what partners can subscribe to

After every scrape, four feed files are regenerated and pushed to GitHub Pages. Anyone can subscribe — no API key, no rate limit. Share the URLs with chamber members, partner sites, and event organisers.

7a. The four feed URLs

Feed	Path under site root	For
iCal (.ics)	feeds/events.ics	Drop into Google Cal / Apple / Outlook as a subscribed calendar
RSS 2.0	feeds/events.rss	Newsletters, partner sites, feed readers
Events JSON	feeds/events.json	Developers building apps / dashboards / integrations
Venues JSON	feeds/venues.json	The 170 curated experiences — stable IDs safe as foreign keys

Site root: `https://liammlrb-eng.github.io/nevada-county-experiences/`

Public docs page: `/feeds/` (under the site root) — subscribe instructions, license, JSON schema, example code. Send this URL to anyone asking "how do I get your events?"

7b. When the feeds refresh

Automatically. `generate_feeds.py` runs as the last step of every scrape (after link checking). It reads `scraper_output/events.json` + the EXPERIENCES list and rewrites the four feed files under `feeds/`. As soon as you push the scrape commit (Section 6), the new feeds go live within ~1 minute (GitHub Pages rebuild). No manual step needed.

7c. License — CC BY 4.0

Subscribers may use, redistribute, remix, and build commercial products on the data provided they credit **Nevada County Experience** with a backlink to the homepage. Each event record also carries a `source` field naming the original publisher (KVMR, NCAC, etc.) — partners should preserve that on display whenever practical so the original venue gets credit too.

7d. Sharing with a partner — what to send

Audience / ask	What to send
Newsletter editor wants our events	Send the RSS URL. Their newsletter tool subscribes; events appear automatically.
Partner site wants to embed our calendar	Send the docs page (/feeds/) and the iCal URL. Most CMS calendar widgets accept
Member venue subscribing in their Google Cal	Send the iCal URL. Google Cal → Other Calendars → + → From URL → paste.
Developer / integrator	Send the docs page (/feeds/) — it has the JSON schema, examples, and license no
Someone wants venues on a map	Send the venues.json URL. Each record has lat/lng, name, blurb, tags.

7e. What's in the feeds (filtering rules)

The feeds mirror what the public site shows:

- **Only approved events** — `status = "approved"`. Pending and dismissed never leave the site.
- **Only future events** — anything whose end date is before today drops off automatically.
- **AI fields preferred** — `ai_area`, `ai_venue`, `ai_summary`, `ai_tags` are used when present; raw scraper fields are the fallback.
- **Descriptions capped** at 600 characters in the JSON feed (full description on the source URL).
- **Stable IDs** — every event has a 12-char hash that survives re-scrapes. Safe to use as a key.

8. Troubleshooting

Most common issues and how to fix them.

Symptom	Fix
Server won't start: "python is not recognized"	Python isn't in your PATH. Reinstall Python from python.org and check "Add Python to PATH" du
Server won't start: "Address already in use"	Port 5000 is busy. Either close the previous server window, or run: <font face="Courier">python s
Site loads but Events tab is empty	On the local server: open Manage Database → Events Queue → click Approve All on pending ev
■ Update Now hangs forever	Selenium/Chromium issue. Close the browser, restart server, try again. If persistent, try a single s
■ AI Categorize button is greyed out	No Anthropic API key set. See Section 4a. Restart the server after adding the key.
■ AI Categorize fails with "API error 401"	Invalid API key. Check for typos in scraper/config.py. Generate a new key at console.anthropic.c
■ AI Categorize fails with "rate limit"	Hitting Anthropic API limits. Lower batch size (edit ai_categorize.py: --batch-size 5) or wait an hou
Edits aren't saving	Click the green ■ Save All button after editing — auto-save isn't enabled for the admin table.
Lost an experience after deleting	Browser localStorage keeps a recent backup. Open browser DevTools (F12) → Application → Lo
GitHub Pages not updating after push	Wait 2 minutes (deploy time). Then hard-refresh (Ctrl+F5). If still stale, check repo Settings → Pa
"Permission denied" pushing to GitHub	Your GitHub credentials expired or you don't have write access. In PowerShell: <font face="Cour

9. File map — where things live

Quick reference for what's in the project folder.

```
nevada-county-experience/
■■■ index.html                    ← The whole frontend (everything visitors see)
■■■ server.py                     ← Flask web server (runs locally)
■■■ start_server.bat              ← Double-click to start the server
■■■ scraper/
■   ■■■ event_scraper.py          ← Main scraper (orchestrates all sources)
■   ■■■ ai_categorize.py          ← AI cleanup (Claude Haiku)
■   ■■■ auto_tagger.py            ← Keyword-based pre-tagging
■   ■■■ generate_feeds.py         ← Builds feeds/ outputs (runs after each scrape)
■   ■■■ config.py                 ← API KEYS — never committed to GitHub
■   ■■■ sources.json              ← Scraper source URL list
■   ■■■ site_scrapers/            ← One file per source (or per platform)
■       ■■■ base.py               ← Shared EventScraper class + RSS autodiscovery
■       ■■■ kvmr.py               ← Tribe Events RSS
■       ■■■ ncac_calendar.py     ← Trumba JSON
■       ■■■ eventbrite_nevada.py ← React via Selenium
■       ■■■ nevada_city_chamber.py
■       ■■■ gv_chamber.py
■       ■■■ the_union.py
■       ■■■ center_for_arts.py
■       ■■■ miners_foundry.py
■       ■■■ fairgrounds.py        ← Saffire JSONP (Nevada County Fairgrounds)
■       ■■■ tribe_events.py       ← The Events Calendar REST (Crazy Horse)
■       ■■■ woocommerce.py        ← WooCommerce Store API (Curious Forge)
■       ■■■ shopify.py            ← Shopify products.json (Wolf Craft)
■       ■■■ squarespace_events.py ← Squarespace events JSON (Golden Era)
■       ■■■ nevada_theatre.py     ← MEC plugin (disabled — see event_scraper.py)
■       ■■■ go_nevada.py          ← disabled (Cloudflare 403s)
■       ■■■ go_nevada_festivals.py ← disabled (Cloudflare 403s)
■■■ scraper_output/
■   ■■■ events.json               ← The live event database (committed)
■   ■■■ candidates.json           ← Scraper debug data (gitignored)
■   ■■■ snapshots/                ← Rendered HTML for selector debug (gitignored)
■■■ feeds/
■   ■■■ index.html                ← Public docs page for partners (/feeds/)
■   ■■■ events.ics                ← iCal feed — calendar app subscribe
■   ■■■ events.rss                ← RSS 2.0 — newsletters, partner sites
■   ■■■ events.json               ← JSON — developer integrations
■   ■■■ venues.json               ← The 170 curated experiences
■■■ demo_pitch.pdf                ← County presentation deck
■■■ operator_guide.pdf            ← This file
■■■ generate_demo_pdf.py          ← Source for demo_pitch.pdf
■■■ generate_operator_guide.py    ← Source for this file
■■■ CLAUDE.md                     ← Project context for future contributors
■■■ .gitignore                    ← What never gets committed
```

8a. The four files you'll touch most

File	When you touch it
index.html	Edit experiences via Manage Database (admin panel) — never edit by hand
scraper_output/events.json	Updated automatically by scraping — commit + push after scrapes
scraper/config.py	API keys — edit once during setup, never commit
scraper/sources.json	Add/disable scraper sources — edit via ■ Scraper Sources tab

8b. Logs & diagnostics

When something's wrong, look here:

Where	What it tells you
Server console window	Live request log + Python errors. Keep visible while debugging.
Browser DevTools console	F12 → Console tab. Frontend errors and warnings.
scraper_output/snapshots/	When a scraper fails: <font face="Courier">python scraper/event_scraper.py --discover --s

10. Annual tasks & emergency contacts

10a. Once-a-year checklist

- **Spring**: walk through every experience entry — verify hours, URLs, and that the venue still exists. Use Manage Database → Experiences. Budget 4-6 hours.
- **Spring**: review all 9 vibes — are the photos still working? Are the taglines still accurate? Edit if needed.
- **Summer**: pre-festival-season scrape + AI categorize + manual review. Push to GitHub.
- **Fall**: review the Suggested Experiences — are the published itineraries still relevant? Update photos and stops.
- **Anytime**: domain renewal — set a calendar reminder 90 days before expiry.
- **Anytime**: backup test — once a year, restore events.json from a tarball backup to a test folder, verify the file loads.

10b. Emergency contact ladder

When something breaks and you need help, in order:

- **1. Try Section 7 (Troubleshooting)** first — covers 80% of issues
- **2. Check the GitHub repo Issues tab** — has anyone reported this?
- **3. Original developer / contractor** — keep their email pinned
- **4. County IT** — for server-level / hosting issues
- **5. Anthropic support** — for AI billing / API key issues (support@anthropic.com)
- **6. Hosting provider support** — DigitalOcean / Linode / etc., for server downtime

10c. Before calling for help — gather this

Info	Why it helps
What you tried	List the steps that led to the problem
Exact error text	Copy-paste from console window or browser. Screenshot if needed.
When it started	After a scrape? After a push? After Windows update?
Which environment	Local server? GitHub Pages? Just the admin panel?
Last successful action	"Scrape worked yesterday at 3 PM" — helpful baseline

Final note

"Most problems with this platform aren't crises — they're small gaps in process. A scrape fails, a tag goes stale, a photo URL breaks. The system is designed to keep working even when individual pieces don't. Don't panic; check Section 7; ask for help if needed."